

Python Fundamentals – Capstone Project

Customer Support Ticket Analyser

Project by:

KALAIVANI BASKAR

**Aspiring Data Analyst | SQL |
Excel | Power BI | Python**

Chennai, India

niva1303@gmail.com |
+91 8675107979

LinkedIn: [in/kalaivani-baskar](https://www.linkedin.com/in/kalaivani-baskar)

GitHub: [GitHub_Kalaivani](https://github.com/Kalaivani)

Project Overview:

The **Customer Support Ticket Analyser** is a Python-based project developed to organize, clean, and analyse customer support ticket data. The project demonstrates the use of Python concepts such as **dictionaries, lists, loops, conditional statements, functions, string methods, and sets** to identify useful patterns in customer support issues.

Data Cleaning:

The issue descriptions were standardized before analysis. The cleaning process included converting text to lowercase, removing punctuation, eliminating unnecessary spaces, and replacing shorthand such as "ok" with "okay". This made the ticket descriptions consistent and suitable for keyword and text analysis. These are the specific cleaning operations required by the assignment.

Key Findings:

Analysis of the **10 original customer support tickets along with 3 newly added tickets** produced the following results:

- **Priority Analysis:** 6 tickets were High priority, 3 were Medium priority, and 4 were Low priority.
- **Keyword Analysis:** "poor" appeared in 3 tickets, "good" in 3 tickets, "slow" in 2 tickets, and "excellent" in 1 ticket.
- **Longest Issue Description:** Ticket No. 2, submitted by **Meera**, had the longest cleaned issue description: "slow response very poor service", containing **5 words**.
- **Unique Word Analysis:** The cleaned descriptions contained **32 unique words**, showing a mixture of positive terms such as "good", "great", "excellent", "helpful", and "satisfied", as well as issue-related terms such as "poor", "slow", and "not".

Insights:

The priority analysis shows that **High-priority tickets form the largest single priority group**, with 6 out of 13 tickets. The keyword analysis identifies both positive and negative service-related language. Words such as "good" and "excellent" indicate positive support experiences, while "poor" and "slow" highlight areas where service quality or response time may require attention.

Conclusion:

The project successfully demonstrates how basic Python programming can be used to transform unstructured customer support descriptions into useful information. By cleaning ticket descriptions and analysing priorities, keywords, word counts, and unique words, the system provides a simple way to identify recurring customer concerns and support-service patterns. The project also provides a foundation that could later be extended to larger datasets and more advanced customer-support analysis.